Practical : 4	Implementation and Time analysis of factorial program using iterative and recursive method
Date : 15/07/2026	

Aim: To implement and compare **iterative and recursive methods for calculating the factorial number**.

Objective :-To implement programs to calculate the factorial of a given number in C++ using both iterative and recursive methods, and to analyze and compare their respective time and space complexities.

A. Iterative Factorial Algorithm

1. Start.
2. Read n.
3. Initialize result = 1.
4. Run a loop from 1 to n.
5. Multiply result by the current value.
6. Return the result.
7. Stop.

B. Recursive Factorial Algorithm

1. Start.
2. Read n.
3. If n <= 1, return 1.
4. Otherwise calculate n × factorial(n-1).
5. Continue until the base case is reached.
6. Return the result.
7. Stop.

Program:

```
#include <iostream>

using namespace std;

// 1. Iterative Method

long long factorialIterative(int n) {
```

```
long long result = 1;
for (int i = 1; i <= n; i++) {
result *= i;
}
return result;
}
// 2. Recursive Method
long long factorialRecursive(int n) {
// Base Case
if (n <= 1) {
return 1;
}
// Recursive Case
return n * factorialRecursive(n - 1);
}
int main() {
int num;
cout << "Enter a positive integer: ";
cin >> num;
```

```
if (num < 0) {
cout << "Factorial of a negative number is not defined."
<< endl;
}
else {
// Iterative Method
long long iterativeResult =
factorialIterative(num);
cout << "\n--- Iterative Method ---" << endl;
cout << "Factorial of " << num
<< " is: " << iterativeResult << endl;
// Recursive Method
long long recursiveResult =
factorialRecursive(num);
cout << "\n--- Recursive Method ---" << endl;
cout << "Factorial of " << num
<< " is: " << recursiveResult << endl;
}
return 0;
}
```

Output:

```
Output

Enter a positive integer: 10

--- Iterative Method ---
Factorial of 10 is: 3628800

--- Recursive Method ---
Factorial of 10 is: 3628800


=== Code Execution Successful ===
```

Analysis:

Time Complexity Analysis

Let n be the input number for which we need to calculate the factorial.

Iterative Method:

- The iterative function executes a simple for loop that runs exactly **n** times to multiply the values from **1 to n**. Since there are no nested steps, the execution time grows linearly with the input size.

Time Complexity = O(n)

Recursive Method:

- The recursive function calls itself continuously, decremented by 1 at each step:
- **T(n) = T(n - 1) + O(1)**
- This sequence continues until it hits the **base case (n <=1)**. This creates a total chain of **n** recursive function calls, resulting in a linear growth rate.

Time Complexity = O(n)

Space Complexity Analysis

Iterative Method:

- The loop uses only a few fixed-size scalar variables (result, i) to run. The amount of memory occupied stays completely constant and independent of the input value n.

  Space Complexity = O(1)

Recursive Method:

- Every time a function calls itself, a new stack frame is pushed onto the system call stack to store the current state and return address. For an input of size n, the system must hold n active stack frames simultaneously before hitting the base case and unwinding.

  Space Complexity = O(n)

Conclusion :-

While both methods successfully calculate the factorial of a number in linear time O(n), their system overhead profiles are vastly different:

1. The **Iterative Method** is highly memory efficient. It works with a constant auxiliary footprint of O (1) space, making it safe from stack exhaustion limits.
2. The **Recursive Method** is mathematically expressive and maps directly to the recursive definition of factorials. However, it requires linear auxiliary memory space of O (n) due to the call stack overhead. For large inputs, this makes recursion prone to a **stack overflow** crash.